# MCS-DOS 2.1

A DOS shell
for the Commodore 64.

## User's Guide and Reference



Published by MCS Labs

© 2026

# Table of contents

**Table of contents** 2

**Introduction** 5

**1. Features and limitations** 6

**2. Development notes** 7

**3. Contents of the cartridge** 7

**4. Running MCS-DOS** 8

Before you start ... 8

Startup ... 8

Boot configuration ... 8

**5. Using the shell** 10

The command prompt ... 10

Running programs ... 10

Using Commodore BASIC ... 10

Devices ... 11

Disk images ... 11

Directories and partitions ... 11

Cartridge storage and persistence ... 12

Redirection and piping ... 12

Theming ... 12

**6. Command reference** 13

ATTRIB ... 13

BASIC ... 14

BEEP ... 14

CD ... 14

CHDIR ... 15

CHKDSK ... 15

CLS ... 16

COPY ... 16

DEL ... 17

DELETE ... 18

DIR ... 18

DISKCOPY ... 19

DISKID ... 20

DISKINIT ... 20

ECHO ... 20

EDIT ..... 21
ERASE ..... 22
FIND ..... 22
FORMAT ..... 23
HELP ..... 24
KEY ..... 24
LABEL ..... 25
MD ..... 25
MEM ..... 27
MKDIR ..... 27
MOUNT ..... 27
MOVE ..... 28
PAUSE ..... 29
PRINT ..... 29
RD ..... 29
REBOOT ..... 30
REM ..... 30
REN ..... 31
RENAME ..... 31
RMDIR ..... 31
RUN ..... 31
SET ..... 32
SPLASH ..... 33
SYSINFO ..... 33
TAPECOPY ..... 34
TYPE ..... 34
UNMOUNT ..... 35
VER ..... 36
VERSION ..... 36
VOL ..... 36

**7. Environment variables 37**

The MCS-DOS environment ..... 37
CHARSET ..... 37
COLOR ..... 38
DIRCMD ..... 38
DRIVEIDS ..... 39
PROMPT ..... 39

**8. Output redirection** **40**

**9. Batch scripting** **41**

Creating and running batch files ..... 41

Batch processing ..... 41

Example ..... 41

AUTOEXEC.BAT ..... 42

**10. Changelog** **43**

**11. About the author** **46**

**Endnotes** **47**

# Introduction

MCS-DOS is a MS-DOS-inspired operating environment designed to work on a Commodore 64, Commodore 64 Ultimate, Commodore 128 in C64 mode, or on emulators [1]. No RAM expansions or CPU upgrades are needed. It provides familiar prompts, commands and syntax to perform disk and file operations. You can run programs just as you would on a stock setup, and the BASIC environment is always just one command away. Rudimentary batch scripting is also supported.

MCS-DOS was created for fun and novelty, and the author makes no claims as to its real-life usefulness. The software is distributed free of charge and licensed under the MIT license.

MS-DOS is property of Microsoft Corporation. No copyright infringement is intended.

Source code and releases are available on GitHub: https://github.com/mvanwijk83/mcs-dos. Code contributions, bug reports, feature requests and discussions are welcome.

## License

Copyright (C) 2026 MCS Labs

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

# 1. Features and limitations

MCS-DOS lets you use your Commodore 64 in a command-line driven manner, with a classic DOS look and feel. It is *not* an (MS-)DOS clone or emulator; you cannot use it to read PC disks and files, let alone execute MS-DOS programs. MCS-DOS takes a workable subset of DOS features and adds a few of its own, offering a genuine DOS-like user experience tailored for the Commodore 64.

It is also a command shell rather than a full operating system, as it does not control the Commodore 64 at a low level; it still relies on the Commodore 64 KERNAL for system services. Nevertheless, it can certainly be utilized as a full-fledged alternative operating *environment*, supporting all common file and disk operations. If you enjoy DOS-style command line interfaces, you will be right at home here.

The main features are:

- Many familiar DOS commands and utilities are implemented.
- Configuration via `AUTOEXEC.BAT` and `CONFIG.SYS`.
- Some batch file support.
- Supports 1541, 1541-II, 1570, 1571, 1581, CMD-FD/HD, SD2IEC and SoftIEC (Ultimate).
- Create and mount disk images on supported storage devices.
- Drives can be optionally displayed and addressed with DOS-style drive letters.
- Simple built-in text editor.
- Run programs and use BASIC, with return to the MCS-DOS shell (under some conditions).
- Quality-of-life features such as command history. filename auto-completion, and key bindings.
- Environment variables.
- Output redirection to files.
- File printing.
- Command help easily accessible from the shell (via `/?` and the `HELP` command).
- Customizable shell including command prompts, colors and custom character sets.

Some notable limitations and omissions include:

- No fastload or other disk access optimization routines are implemented. Standard disk transfer speeds apply, unless ROM replacements such as JiffyDOS or DolphinDOS are used.
- Native programs launched from the shell cannot always return to the MCS-DOS shell. This is not only because many don't "exit" as such, but also because MCS-DOS relies on a RAM-resident return hook that may get overwritten by the software.
- Batch files do not support variables, conditionals, labels, or parameters.
- No piping (`|` syntax) or input redirection (`<`).
- Due to the shell's mixed-case mode, half of PETSCII's graphics characters do not display correctly, and cannot be typed.
- No internationalization.
- No REU support.

Future versions may address these and other shortcomings where possible.

**Note on compatibility:** As this is a one-man hobby project, not every possible combination of hardware models, variants, KERNALs, and peripherals can be reliably tested, even where compatibility is claimed or implied. Where necessary and possible, verification is done through emulation instead, whether manual or automated.

# 2. Development notes

MCS-DOS started as a simple throwaway project for personal entertainment, motivated by a geeky impulse to see a believable DOS-style environment running on the C64, supporting only a few basic commands at first. Once things started to look genuinely usable, it became a serious project, and a continuing effort to keep improving and polishing it.

The software was written in C and compiled with Oscar64 on Windows 11. It was tested on a C64 Ultimate and in the VICE emulator. Automated unit and regression testing is implemented with Node.js.

**AI disclosure:** OpenAI's Codex has been extensively used to assist with implementation, review, automated unit/regression testing, and build tooling. The software as a whole, however, is the product of hundreds of hours of research, planning, design, documentation, and manual testing and revision. Despite machine assistance, MCS-DOS is very much a product of human intellect, creativity and effort, not merely "slop".

# 3. Contents of the cartridge

MCS-DOS 2.1 is distributed as an EasyFlash cartridge image. The following files are bundled with the software:

| | |
|---|---|
| `AUTOEXEC.SAMPLE` | Example of an `AUTOEXEC.BAT` file, demonstrating some basic customization options. Rename the file to enable. |
| `CGA.CPI` | A custom character set reproducing the 8x8 IBM CGA font. For more information about using custom fonts, see chapter 7. |
| `CHANGELOG.TXT` | A version-by-version list of changes to MCS-DOS. |
| `CONFIG.SAMPLE` | Example of a `CONFIG.SYS` file, demonstrating additional boot configurations settings. Rename the file to enable. |
| `LICENSE.TXT` | The MIT license as it applies to MCS-DOS. |
| `MANUAL.TXT` | An abridged plain text version of this manual. |
| `VGA.CPI` | A custom character set reproducing the 8x8 IBM VGA font. |

# 4. Running MCS-DOS

## Before you start

If you intend to use a physical EasyFlash or compatible product, transfer the MCS-DOS image to the cartridge first. Refer to your hardware product's documentation for further instructions.

If you're using an Ultimate, you can copy the .CRT to SD or USB storage, and run it directly from the file browser. If you want it to run at machine boot, copy it to the internal flash and set it as the default cartridge under Memory & ROMs. In VICE, use File → Attach cartridge image, or Preferences → Settings → Cartridges to set it as the default startup cartridge.

Note that MCS-DOS uses KERNAL routines for file and disk access, with all the slowness it entails – no speeder code in any form has been implemented. For improved performance, consider using ROM replacements such as JiffyDOS or DolphinDOS.

## Startup

Upon booting the software, you are presented by a logo splash screen and some basic information about your system that MCS-DOS has detected. You can press RESTORE to enter a "safe mode", which will skip processing of the startup files. This can be useful to correct misconfigurations.

Upon entering the command shell, you are presented with a prompt denoting you're on drive "0", which denotes the MCS-DOS cartridge.




*MCS-DOS boot-up sequence: the logo splash screen, system test, and entry to the command shell.*

## Boot configuration

If an `AUTOEXEC.BAT` is present, it will execute before you arrive at the shell prompt. You can use it to apply custom settings to the environment, render custom output, or to run certain commands. It is not present by default and must be created first. See chapter 7 for more information on how to configure and customize the shell via environment variables, and chapter 9 for information about batch files and how they are supported in MCS-DOS.

By default, MCS-DOS looks for `AUTOEXEC.BAT` on the cartridge image. This behaviour can be changed in `CONFIG.SYS`, which is processed before `AUTOEXEC.BAT`. The following directives are available:

| | |
|---|---|
| `BOOTDRV=drive[,...]` | Specify one or more drives where MCS-DOS should seek `AUTOEXEC.BAT`, in that order. For example, `BOOTDRV=8,0` will look for `AUTOEXEC.BAT` on the first disk drive (device 8) first, and then on the cartridge (device 0) if it wasn't found. If `BOOTDRV` is omitted, device 0 is assumed. |
| `LDAUTOEX=0\|1` | Whether `AUTOEXEC.BAT` should be executed at all. `LDAUTOEX=0` will disable it; `BOOTDRV` would then not take effect either. The default value when omitted is 1 (i.e. load and run `AUTOEXEC.BAT`). |

The `CONFIG.SYS` directives are case-insensitive, and whitespace is ignored.

# 5. Using the shell

## The command prompt

See chapter 6 for a list of supported commands. The command interpreter is case-insensitive, and treats spaces before switches as optional. For example, `DIR/W` and `DIR /W` both work. The maximum full length of a command, including parameters, is 127 characters.

You can use CRSR LEFT/RIGHT to move within the command text, while typing inserts. The last ten commands are retained in a history buffer; use CRSR UP/DOWN to cycle through them.

When specifying paths and/or file names as command parameters, double-quote them if they contain spaces or a slash, e.g.: `COPY "MY DATA" B:`.

File name autocompletion can be invoked with the Commodore key. If there are no preceding characters, it will start from the first alphabetical entry on the active disk. Existing input will be completed if there are exact matches; e.g. typing `MAN` followed by pressing the Commodore key will bring up `MANUAL.TXT` if that's the first match in the current directory. There may be a small initial delay as MCS-DOS builds the directory cache. Rely on this feature especially if file names contain PETSCII graphics characters that do not display correctly in the shell's mixed-case mode; the autocomplete internally preserves the byte-correct original filename.

The appearance of the command prompt can be configured with `SET PROMPT` in `AUTOEXEC.BAT` to be applied at startup. See chapter 7 for more details. The default is to show the current IEC device number and, if applicable, the current path.

RUN/STOP aborts operations when possible (think of it as the `Ctrl+C` equivalent). It can also used to clear current input at the prompt.

## Running programs

Native C64 programs or batch files can be started from the shell simply by typing their name (quoted if necessary). Alternatively, an explicit `RUN` command can be used. Prior to version 2.1, this was the canonical way to start a program in MCS-DOS.

Batch files run fully within the shell and return control after completion. Native program return is not guaranteed. Not only do many programs not actually exit, but MCS-DOS also relies on a RAM-resident return hook, which may get overwritten by the program.

When the shell transfers control to an external program, the current shell state (including settings, history, internal caches, and current path) is saved to the cartridge. Any custom character set that is active at the time will be reset to the C64 ROM default for the duration of the program session. If the program exits and reaches the MCS-DOS return hook, the shell is restored from the saved state. If this fails for some reason, default shell settings will be applied.

## Using Commodore BASIC

The `BASIC` command can be used to enter the C64 BASIC environment. MCS-DOS installs a return wedge that allows you to return to the shell by typing `SHELL` at the BASIC prompt. However, if the memory area starting at `$C000` gets overwritten, this will no longer work. As with native program execution, the MCS-DOS shell is suspended until return.

## Devices

MCS-DOS uses standard C64 device numbers for drives, but also supports DOS-style letter aliases: drive 8 is A, drive 9 is B, etc. Both styles can be used interchangeably. The display convention used by system commands is determined by the `DRIVEIDS` environment variable (see chapter 7). Typing a drive number or letter followed by a colon (e.g. `8:`, `A:`) will select that drive as the current one. The cartridge is identified as drive 0 regardless of C64 or DOS display mode.

Similarly, for printer output both drive numbers 4 and 5 can be specified, or DOS aliases `LPT1` and `LPT2`.

Supported floppy drive models are CBM models 1541, 1541-II, 1570, 1571, and 1581, CMD FD series floppy drives [2], and the CMD HD series of hard drives. This would naturally include clones, compatibles, and emulations of aforementioned models.

SD2IEC is supported as a modern mass storage device, while Ultimate storage is accessible via Software IEC, if enabled. Partitions and subdirectories are implemented; see the "Directories and partitions" section for more details. Long file names (>16 characters) are truncated on Ultimate; it is recommended to rename them to fit the 16 character limit before using them under MCS-DOS, preventing potential name collisions. On SD2IEC, native FAT names are reformatted to 8.3.

Some Datasette (tape) support is present in the form of the `TAPECOPY` command which can be used to transfer files from tape to disk (or even the MCS-DOS cartridge). Program (`PRG`) files of up to 44 kb, and sequential data (`SEQ`) can be transferred, assuming no custom loader is used on the source tape. If a non-standard tape header is identified, `TAPECOPY` will terminate.

## Disk images

Disk images files can be mounted on an SD2IEC device, or to an emulated Ultimate disk drive using the `MOUNT` command (`UNMOUNT` to "eject"). Once mounted, they behave as any other (physical) disk, and regular file and disk operations work.

Supported disk image formats depend on device capabilities: D64, D41, D71, D81 and DNP on SD2IEC, while on an Ultimate, D64, D71, D81, G64 and G71 should work.

See the `MOUNT` and `UNMOUNT` command documentation for more information.

## Directories and partitions

Some drive types support partitions and subdirectories. MCS-DOS abstracts both into a unified tree structure as if they're all part of the same logical file system. Partitions are navigated and managed like ordinary subdirectories, using directory commands (`CD`, `MD`, `RD`) and represented as part of a drive path (e.g. `\PART01\SUBDIR\FILE`). Some limitations may apply depending on drive type. No manual partition switching is needed; this is handled automatically and transparently by MCS-DOS.

Partition numbers can be used instead of names; e.g. `\@1`. Unnamed partitions are displayed and can be referred to with this alias.

As is generally the case in DOSes, MCS-DOS uses the backslash (\) as a path separator. Because a C64 keyboard does not include this symbol, you must use the pound key (£) to type it. MCS-DOS modifies the character set to substitute the pound symbol for a backslash.

**Commodore 1581:** does not support “true” subdirectories, but 1581 partitions can be nested and derive from a common root. As such, they map seamlessly to a directory tree structure. When creating new 1581 partitions (via `MD`), you must specify the size: the minimum size is 120 sectors, and the total size must be a multiple of 40.

**CMD FD/HD:** drives employ a “flat” partition structure; partitions can therefore not contain other partitions. Each of these partitions can be independently formatted with either the native CMD file system, or emulate 1541, 1571 or 1581 disks. MCS-DOS “simulates” a root directory by treating the partitions as first-level subdirectories, which each can contain further subdivisions, depending on how the partition was formatted. Native CMD partitions can contain subdirectories, but partitions cannot be created, modified or removed with MCS-DOS commands; you must use FD-TOOLS or HD-TOOLS instead.

**SD2IEC:** partitions and subdirectories are fully supported, with the former appearing as first-level subdirectories. Note that although SD2IEC treats disk images as subdirectories at the protocol level, using `CD` with them does not work; in MCS-DOS, you must explicitly (un)mount them.

**Ultimate:** storage is accessible via Software IEC, which must be enabled and configured from the Ultimate menu (`F2` → Software IEC Settings). SD, USB, internal flash and temp storage are shown as first-level subdirectories. SoftIEC mounts explicitly set via the file browser are not detected.

## Cartridge storage and persistence

64,000 bytes of user-writable storage is available on the cartridge image. It is the default location for your startup configuration files and where the bundled documentation resides. However, nothing stops you from storing anything else you may want to keep there. Indeed, if your boot script references resources not bundled with MCS-DOS, you might want to copy them there instead of relying on an external disk.

A custom, append-only journaling file system is implemented to allow for fast file operations while reducing physical flash writes. It however requires occasional compaction to reclaim space from stale metadata records and deleted content. This happens automatically, which is noticeable as a slowdown during writes when it occurs. Compaction can also be forced via the `CHKDSK /C` command.

Changes to the writable area of the cartridge image persist automatically on a real EasyFlash cartridge. In VICE, you may have to enable writeback settings. On an Ultimate, you need to press `F1` in the Ultimate menu, go to Power & Reset, and select Save Cartridge. This updated image must be loaded next time you use MCS-DOS. Some flash cartridges will require manual saving as well.

## Redirection and piping

Command output redirection to file is supported. See chapter 9 for more extensive information. Input redirection, printer redirection and piping (`|` syntax) are not supported.

## Theming

The appearance of the shell (colors, character set, prompt format, drive identifiers) can be customized via environment variables. See chapter 7 for more information.

# 6. Command reference

The MCS-DOS command set is largely a subset of MS-DOS, but a few unique commands and command options are available as well to support Commodore 64-specific features, or simply because they were deemed useful.

A list of supported commands can be viewed from within the shell by typing `HELP`. Help on a specific command can be viewed by typing `HELP command`, or by using the `/?` switch with that particular command. For example, to view help on `COPY`, the following commands are equivalent:

```
HELP COPY
COPY /?
```

Wildcards `*` and `?` work with most file operations.

In most cases, the output of commands will paginate if they exceed the vertical screen space.

For clarity and readability, command names are displayed in upper case in this chapter. Note however that the actual command shell is case-insensitive, and commands will also work when typed in lower or even mixed case.

## ATTRIB

Displays or changes file attributes.

**Syntax:**

```
ATTRIB [+L | -L] [[drive:]filename]
```

**Description:**

The CBM-DOS file system implements a “file lock” bit which prevents a file from being deleted or overwritten. If the bit is set, any such file operation will fail or be prevented.

Older MCS-DOS versions (1.0 and 1.01) used the letter `R` instead of `L` to mirror the MS-DOS read-only attribute. This has been changed to more accurately reflect CBM-DOS behavior and semantics. No backwards compatibility has been implemented.

If `ATTRIB` is run without attribute modifiers or a filename, a list of all files within the current directory is displayed with their respective attributes.

**Examples:**

Set and remove a lock to a file:

```
ATTRIB +L REPORT.TXT
ATTRIB -L REPORT.TXT
```

View attributes within the directory, of an individual file, or a group of files (using wildcards):

```
ATTRIB
ATTRIB REPORT.TXT
ATTRIB *.TXT
```

# BASIC

Starts the Commodore 64 BASIC environment.

**Syntax:**

```
BASIC
```

**Description:**

Using the BASIC command will suspend the MCS-DOS shell and transfer control to the Commodore 64 BASIC environment. MCS-DOS installs a "wedge" at RAM address `$C000`, which allows you to return to the MCS-DOS shell from BASIC by typing `SHELL`. This functionality remains available as long as no program executed within the BASIC environment overwrites that area.

The reserved area lives "above" the reserved BASIC RAM; the full 38911 bytes workspace therefore remains available for BASIC programs.

Prior to entering BASIC, the current shell state (including configuration, history, and internal caches) is saved to the cartridge, and restored if the MCS-DOS return hook is successfully reached. If restoration fails for some reason, default shell settings will be applied.

# BEEP

Plays a SID beep.

**Syntax:**

```
BEEP [duration]
```

**Description:**

`BEEP` may be useful in batch programs to alert a user. The duration is in milliseconds; the default is 200.

**Example:**

Play a beep with a duration of half a second:

```
BEEP 500
```

# CD

Displays or changes the current directory or partition.

**Syntax:**

```
CD [drive:][path] [/D]
CHDIR [drive:][path] [/D]
```

**Description:**

The `CD` or `CHDIR` command changes the current working directory on drives that support subdirectories and/or partitions. See chapter 5, "Directories and partitions", for more information.

Both relative and absolute paths work, as well as dot notation (`.` for the current directory and `..` for the parent). You can also change the directory on a different drive, but you must specify the `/D` switch to also change *to* that drive. If no parameters are given, `CD` will display the current path.

The following switches are available:

`/D` Changes drive in addition to changing current directory for a drive.

**Examples:**

Change to a specific subdirectory on the same drive:

```
CD DEMOS
CD \DEMOS\X2024
```

Change the directory on a different drive and switch to that drive:

```
CD A:\GAMES /D
```

Change to the root directory of the current disk:

```
CD \
```

Change from a subdirectory to its parent directory:

```
CD ..
```

## CHDIR

Alias of `CD`.

## CHKDSK

Checks a disk and returns a status report.

**Syntax:**

```
CHKDSK [drive:] [/V] [/C]
```

**Description:**

The `CHKDSK` command shows the status of the current or specified disk. It also provides a validation feature and can perform an on-demand compaction of the cartridge's user-writable storage.

When run without switches, `CHKDSK` displays the total amount of disk space, the space allocated for the files currently on the disk, directory/partition overhead, and the amount of free space. These are displayed in bytes, but are directly calculated from CBM-DOS "blocks"; i.e., units of 256 bytes. Disk size and remaining space are also displayed in blocks, except when querying the cartridge storage; blocks are not a meaningful entity here. Finally, the drive model and drive identifiers (device number and DOS letter) are reported.

Beware that on 1581 drives, `CHKDSK` includes space reserved for partitions in the file allocation count. This can seem confusing when you encounter a large figure, but few or no files are present, or only small ones. You are in fact seeing the full partition sizes included in the count: this is a limitation of how the 1581 reports its file allocation figures.

On CMD drives, `CHKDSK` only works on a per-partition basis, and does not work on native CMD partitions. It is wholly unsupported on SD2IEC or Software IEC devices, although it does work on disk images mounted from those drives.

The following switches are available:

| | |
|---|---|
| `/V` | Executes the CBM-DOS `VALIDATE` command on a floppy disk. It reconciles the block availability map (BAM) with the actual directory contents, freeing any blocks no longer in use by files and deleting all unclosed files. This functionality is not available on drive 0 (the MCS-DOS cartridge) and mass storage drives. On 1581 and CMD drives, this validates all existing partitions on the drive. |
| `/C` | "Compacts" the user-writable cartridge storage area if required. This removes stale metadata records and removes file data marked as deleted. Compaction normally occurs automatically when the cartridge file system is determined to be "full" during a write. It is only applicable to the MCS-DOS cartridge. |

**Examples:**

Output a status report of the current disk, or a specific one:

```
CHKDSK
CHKDSK 8:
CHKDSK A:
```

The output can also be redirected to a file:

```
CHKDSK > STATUS.TXT
```

Validate the current or a specific disk:

```
CHKDSK /V
CHKDSK 8: /V
```

Compact the cartridge file system:

```
CHKDSK /C
```

## CLS

Clears the screen.

**Syntax:**

```
CLS
```

**Description:**

Running `CLS` will clear the screen of all currently visible output, leaving a blank screen with only the prompt and cursor. Aside from letting you work in the shell with a "clean slate", it may also be useful in batch scripts.

## COPY

Copies a file to another location, or combines files on the same disk.

**Syntax:**

```
COPY source [destination] [/P]
COPY source1+source2[+...] destination
```

### Description:

Use COPY with a source and destination to transfer files to another drive, or to create a duplicate under another name. Both can include paths or a drive specifier. Omitting *destination* will copy the file (or files matching a wildcard) to the current location.

COPY can also be used to concatenate multiple different files into one single destination file, but beware that due to CBM-DOS limitations, the combined source and destination string cannot exceed 40 characters.

If an existing file is overwritten, you will be prompted for confirmation.

The following switches are available:

| | |
|---|---|
| /P | Suppresses the confirmation prompt to overwrite an existing file. |

### Examples:

Copy a file or file(s) from one drive to another:

```
COPY MYDATA 9:
COPY A:DIARY*.TXT B:
```

Copy a file or file(s) from another path or drive to the current location:

```
COPY /DATA/001.DAT
COPY B:/DATA/*.DAT .
```

Create a copy of a file under another name:

```
COPY REPORT.TXT REPORT2.TXT
```

Concatenate several files into one single combined file:

```
COPY 01+02+03+04 ALL.DAT
COPY *.TXT+REPORT* COMBINED.TXT
```

## DEL

Deletes one or more files.

### Syntax:

```
DEL [drive:]filename [/P]
DELETE [drive:]filename [/P]
ERASE [drive:]filename [/P]
```

### Description:

A single file or a group of files matching a wildcard can be deleted (“scratched” in traditional C64 terminology) from the current or specified drive. By default, MCS-DOS prompts for confirmation to proceed with deleting a file. When multiple files are deleted through the use of a wildcard, confirmation is asked for each matching file. Confirmation can be disabled with the /P switch.

The following switches are available:

| | |
|---|---|
| /P | Suppresses the confirmation prompt to delete a file. |

**Examples:**

Delete a single file on the current drive:

```
DEL MAIN.ASM
```

Delete a single file on a different drive:

```
DEL A:HISCORES
```

Delete a group of files matching a wildcard:

```
DEL *.TXT
DEL VACATION*
```

Delete a group of files matching a wildcard, but suppress the confirmation prompt:

```
DEL *.TXT /P
```

## DELETE

Alias of `DEL`.

## DIR

Displays a list of files on a disk.

**Syntax:**

```
DIR [drive:][pattern] [/B] [/L] [/O[sort]] [/Ttype] [/W]
```

**Description:**

The `DIR` command when typed without any parameters will produce a list of all files and subdirectories (where supported) on the current drive. A drive and/or a pattern (filename or a wildcard expression) will output a corresponding listing.

The directory listing shows file name, type and size. The `DIR` header lists volume label (disk name) and disk ID. In the footer, the number of files with their combined size are shown, as well as the number of subdirectories, and the amount of space free on the drive. File size amounts are given in both bytes and disk blocks (abbreviated `bl` in the command output). Note that file sizes reflect *allocated* sizes as reported by the file system, rather than actual file content size. Different drive types may report different sizes based on their storage and allocation schemes.

Contrary to classic DOS, dates and times are not displayed because the CBM-DOS file system does not store them (the C64 does not have a real-time clock, unless via third-party hardware expansions). File timestamps on CMD, SD2IEC or Ultimate drives will not be displayed either.

The file type column lists the native CBM types `PRG`, `DEL`, `REL`, `SEQ` and `USR`. Additionally, supported disk image types are distinguished: `D64`, `D41`, `D71`, `D81`, `DHD`, `DNP`, `G64` and `G71` [3]. Directories and partitions are indicated by `<DIR>`. Partition filesystem types are also reported if applicable.

The following switches are available:

`/A` Displays files with specified attributes.
- `D` Directories [4]
- `L` Locked files

A minus sign (-) can be prepended to invert the selection (i.e. exclude the specified type).

`/B` Uses bare format: lists file names only with no further file and directory statistics.

`/L` Displays file names in lower case.

`/N` Displays native FAT file names (Ultimate). File metadata is omitted.

`/O[sort]` Lists files in sorted order. A sort option can be appended (default is `N`):

- `N` Alphabetically by name.
- `T` Alphabetically by type.
- `S` By size, smallest first.

A minus sign (`-`) can be prepended to the sort option to reverse the order.

`/S` Displays files in all subdirectories (paths and file names only).

`/T[type]` Lists files of a given type: `D=DEL, P=PRG, R=REL, S=SEQ, U=USR`. The `I` argument will list mountable disk images (see `MOUNT`).

`/W` Displays file names in wide format (two columns, no file statistics).

**Examples:**

Show the full directory of the current drive, or that of a different one:

```
DIR
DIR 8:
```

Show only file names, in lower case:

```
DIR /B /L
```

Show files ordered by size (largest first), and preserve the first entry:

```
DIR /O-SF
```

Show only program (PRG) files:

```
DIR /TP
```

Show files matching a wildcard pattern:

```
DIR *.TXT
DIR A:DATA?
```

Output can be redirected to a file:

```
DIR A:*.TXT > TEXTFILES
```

## DISKCOPY

Copies the contents of one disk to another.

**Syntax:**

```
DISKCOPY source: destination:
```

**Description:**

The source and destination drives must be of the same model; e.g. it is possible to duplicate a disk between two 1541 drives, or two 1571s, but from a 1541 to a 1571 or vice versa is currently not

supported. Both drives must be switched on. Due to memory limitations, it is also not possible to duplicate a disk using only a single drive with a disk swap in between.

**Example:**

Copy the contents of drive A (8) to B (9):

```
DISKCOPY A: B:
```

## DISKID

Changes the drive's two-character ID.

**Syntax:**

```
DISKID [drive:] [id]
```

**Description:**

Standard Commodore disks have a two-character alphanumeric ID. The disk ID does not have a set purpose, but it may be used as e.g. a serial number, or by multi-disk software to identify whether the expected disk is inserted.

If no drive is specified, the current drive is assumed. If the ID is omitted, `DISKID` will prompt for it. `DISKID` does not work on the MCS-DOS cartridge.

**Examples:**

Change the ID of the current disk but have MCS-DOS prompt for the name:

```
DISKID
```

Change the ID of a disk in drive A (8) to "01":

```
DISKID A: 01
```

## DISKINIT

Resets disk drive state and forces a re-read of the directory.

**Syntax:**

```
DISKINIT [drive:]
```

**Description:**

This command executes the CBM-DOS `INITIALIZE` command. It forces the disk drive to re-read the disk's block allocation map (BAM) into its internal memory. This may sometimes be needed when disks with the same ID (see `DISKID`) are swapped, leaving the disk drive's cache in an inconsistent state. If the drive parameter is omitted, the current drive is assumed.

`DISKINIT` is not supported nor required on CMD drives and mass storage devices.

# ECHO

Displays messages, or turns command-echoing on or off.

**Syntax:**

```
ECHO [ON | OFF]
ECHO [message]
ECHO.
```

**Description:**

When a batch file is executing, the default DOS behavior is to display (echo) the command that is being executed. This can be turned off by using the `OFF` option with the `ECHO` command. Similarly, this behavior can be re-enabled with `ECHO ON`. If neither `ON` or `OFF` are specified, the current setting is displayed.

To prevent an `ECHO OFF` command in a batch file from being echoed itself, you can prepend the command with an at-symbol (`@`).

`ECHO` can also be used to display a line of text on the screen, which can be useful in batch scripts. The message cannot contain line-breaks. To display multiple successive lines of text, put several `ECHO` commands after each other. Using `ECHO.` (a dot with no separating space) will force a blank line.

It is possible to omit the line terminator by putting a semi-colon (`;`) at the end of a message. This will cause the following output to continue on the same line. A double semi-colon (`;;`) should be used to display the character literal instead.

**Example:**

A short batch file that demonstrates disabling command echoing, displaying messages, and inserting blank lines:

```
@ECHO OFF
PAUSE Put the destination disk in drive B.
FORMAT B:
ECHO.
PAUSE Put the source disk in drive A.
DISKCOPY A: B:
ECHO Done.
```

# EDIT

Edits one screen of text.

**Syntax:**

```
EDIT [filename]
```

**Description:**

MCS-DOS has a simple file editor that provides a 40 columns by 24 rows surface of editable text. It is intended for basic editing of small text files; it does not handle longer files at this stage. If you need to create or edit larger files, use an external program.

A file name can be specified on the command line to open an existing file, or to create a new file under that name. If no file name was specified, you will be asked for one when saving the file.

EDIT has been designed to behave like the C64 BASIC environment in terms of editing. For example, you can freely use the cursor keys to navigate around the screen, and typing overwrites.

The following edit keys are available:

| | |
|---|---|
| RETURN | Move to the next row. |
| INST/DEL | Delete the previous character. |
| SHIFT + INST/DEL | Insert a space after the current position. |
| CTRL + INST/DEL | Insert a blank line before the current line. |
| CLR/HOME | Move the cursor to the top left corner. |

The bottom row is occupied by a status bar. It shows the current line and column, separated by a colon (e.g. 08:02 when the cursor is on the second column of the eighth row), and the file name if applicable.

When you are done editing, press RUN/STOP. EDIT will ask if you want to save the file, and prompts for a filename if not previously specified. You are asked to confirm when overwriting an existing file. The saved file will contain carriage return line endings, and trailing row spaces and unused bottom rows are trimmed.

## ERASE

Alias of DEL.

## FIND

Searches for a text string in a file.

**Syntax:**

```
FIND [/V] [/C] [/N] [/I] "string" [drive:][path]filename
```

**Description:**

The FIND command looks for the specified string in a single file. The string must be enclosed in double quotes. It then displays any lines it has found that contain the string (or don't, if /V is used). The search is case-sensitive by default.

The following switches are available:

| | |
|---|---|
| /V | Displays all lines *not* containing the specified string. |
| /C | Displays only the count of lines containing the string. |
| /N | Displays line numbers with the displayed lines. Displayed lines will be truncated if necessary. |
| /I | Performs a case-insensitive search. |

**Examples:**

Finds the string "DOS" in the file MANUAL.TXT:

```
FIND "DOS" MANUAL.TXT
```

Displays the number of lines containing a comma in the file NOTES.TXT:

```
FIND "," /C NOTES.TXT
```

Displays all lines *not* containing the string “IN” (case-insensitive) in the file LOREM IPSUM, and shows the line numbers:

```
FIND "IN" /V /N /I "LOREM IPSUM"
```

## FORMAT

Formats a disk.

**Syntax:**

```
FORMAT drive:
```

**Description:**

The `FORMAT` command executes the CBM-DOS disk command `NEW`, which prepares a disk for use with a Commodore 64 disk drive. Any data already contained on it will be irrevocably deleted. You must specify the drive that contains the disk to format. You will be prompted to insert a disk if none is present in the drive yet.

When you format a disk, you will also be asked to provide a volume label (disk name) and two-character disk ID, unless `/V` and/or `/I` switches are used.

Note that `FORMAT` executes the standard CBM-DOS command `NEW`. This utilizes standard track layouts. For example, while it is theoretically possible to format a 1541 disk to use 40 tracks instead of 35, `FORMAT` does not support this; use specialized software instead.

Using `FORMAT` on a CMD-FD requires specifying the `/F` switch to select a supported format type. Formatting a CMD-HD disk is not supported; use the HD-TOOLS utility instead.

The following switches are available:

`/V:label` Specifies the volume label.

`/I:id` Specifies the disk ID.

`/Q` On CBM disks, this performs a “quick format”. This only wipes the block availability map and directory, leaving the original file data present but invisible. New disk writes will overwrite the original data. If selecting this option, `/I` will be ignored. On a CMD disk, it will format a partition.

`/F:format` Specifies one of the following CMD-FD format types:

| | |
|---|---|
| `81` | Unpartitioned 1581 disk (DD, 800 KB). |
| `DD8` | Creates a single 1581 partition (DD, 800 KB). |
| `HD8` | Creates two 1581 partitions (HD, 1.6 MB). |
| `ED8` | Creates four 1581 partitions (ED, 3.2 MB). |
| `DDN` | Creates a single native double density partition (800 KB). |
| `HDN` | Creates a single native high density partition (1.6 MB). |
| `EDN` | Creates a single native extra-high density partition (3.2 MB). |
| `S8` | Creates as many 1581 partitions as will fit. |
| `SW` | Creates the largest possible native partition. |

If multiple partitions are created, the respective partition names will follow the specified volume label (disk name), with the 15th character a space and the 16th character a partition number. FD-TOOLS can be used to further manage the disk layout; MCS-DOS `MD` and `RD` will not work.

**Examples:**

Format a standard Commodore disk in drive 8:

```
FORMAT 8:
FORMAT 8: /Q
```

Format a Commodore disk in drive 8 (A), with a predefined name and ID:

```
FORMAT 8: /V:"WORKDISK" /I:01
```

Format a CMD floppy disk in drive 8 (A) with a maximum-size native partition:

```
FORMAT 8: /F:SW
```

## HELP

Displays help on MCS-DOS commands.

**Syntax:**

```
HELP [command]
```

**Description:**

`HELP` with no command parameter will display a list of MCS-DOS commands. Invoking `HELP` with a command name is equivalent to using the `/?` switch on that command. The help text will usually contain a brief command description, the formal syntax, a list of supported switches, and in some cases some additional usage notes.

**Examples:**

Show a list of MCS-DOS commands:

```
HELP
```

Show help on the COPY command:

```
HELP COPY
```

## KEY

Binds command text to function keys.

**Syntax:**

```
KEY [/Fn[:command]] [/CLEAR[:n]]
```

**Description:**

`KEY` provides a facility to assign commands to the function keys, F1 to F8. By binding command text to a function key, pressing that key on the command prompt will recall that text. Pressing CTRL with the function key will immediately execute it.

MCS-DOS reserves a total space of 256 bytes for all function key bindings.

The following switches are available:

| | |
|---|---|
| `/Fn` | Displays the current binding for the specified function key *n*. |
| `/Fn:command` | Binds a command to function key *n*. |
| `/CLEAR` | Clears all bindings. |
| `/CLEAR:n` | Clears the binding for function key *n*. |

Using KEY without parameters will show all current bindings.

**Examples:**

Bind a command to one or more function keys:

```
KEY /F7:REBOOT
KEY /F1:HELP /F3:DIR /F5:BASIC /F7:EDIT
```

Command switches, parameters, and quotes will work as well:

```
KEY /F1:"DIR /ON"
KEY /F1:"EDIT AUTOEXEC.BAT"
```

Show all bindings, or for a single key:

```
KEY
KEY /F1
```

Clear all bindings, or for a single key:

```
KEY /CLEAR
KEY /CLEAR:1
```

## LABEL

Creates, changes, or deletes the volume label of a disk.

**Syntax:**

```
LABEL [drive:] [label]
```

**Description:**

A "volume label" in DOS is the name of a disk, which can be up to 16 characters in length. It will be displayed as a part of the directory listing shown by DIR, along with the disk ID (see DISKID). The VOL command will display both the label and the ID.

LABEL will prompt for a new name if the *label* parameter was not supplied. Pressing ENTER without specifying a name will delete the current name. The current drive will be assumed if none was specified with the command.

**Examples:**

Change or delete the volume label of the current disk, or a disk in the specified drive:

```
LABEL
LABEL A:
```

Explicitly set the volume label of a disk to "DATA":

```
LABEL DATA
LABEL A: DATA
```

## MAKEDISK

Creates a blank disk image.

**Syntax:**

```
MAKEDISK [drive:][path]filename [/V:label] [/I:ID]
```

**Description:**

`MAKEDISK` can be used to create a new D64, D71 or D81 disk image file on an Ultimate or SD2IEC storage device, with a standard track count. The file name must include an extension that denotes the disk image format. A missing or invalid extension is rejected.

Before image creation, you are prompted for a volume label and disk ID, unless the `/V` and/or `/I` switches are used.

**Examples:**

Create a blank D64 disk image on the current storage device:

```
MAKEDISK WORKDISK.D64
```

Create a blank D71 disk image in a subdirectory on the SD2IEC on drive D (device number 11):

```
MAKEDISK D:\WORKDISKS\20261112.D71
```

Create a blank D64 disk image with predefined volume label and disk ID:

```
MAKEDISK ARCHIVE-2026-11.D64 /V:"NOVEMBER 2026" /I:6B
```

## MD

Creates a directory or partition.

**Syntax:**

```
MD [drive:][path]
MKDIR [drive:][path]
```

**Description:**

The `MD` or `MKDIR` command lets you create a new subdirectory or partition on drives that support them. See chapter 5, “Directories and partitions”, for more information.

On CMD drives, creating new native partitions (i.e. first-level subdirectories) with `MD` is not supported; you must use HD-TOOLS or FD-TOOLS instead.

The 1581’s file system requires that partitions have a size that is a multiple of 40, with a minimum size of 120 sectors. The reserved sectors must also be available contiguously on the disk. Therefore, having enough room free on the disk does not necessarily mean you can also create a partition if that free space is too fragmented. You can specify the partition size with the `/SIZE` switch; if omitted, MCS-DOS will prompt you to specify a size.

The following switches are available:

`/SIZE:n` Specifies the 1581 partition size in sectors (minimum 120, multiple of 40).

**Examples:**

Create a subdirectory in the current directory:

```
MD DOCUMENTS
```

Create a subdirectory “FORMS” in the directory “DOCUMENTS”, residing one level up:

```
MD ..\DOCUMENTS\FORMS
```

## MEM

Displays the amount of used and free memory in your system.

**Syntax:**

```
MEM
```

**Description:**

`MEM` shows the total amount of installed RAM in your computer, and provides a breakdown of current memory usage. Below follows an annotated example of typical output:

| | |
|---|---|
| `65,536 bytes total memory` | The total amount of RAM installed in your system. |
| `29,967 bytes shell and workspace` | The amount of RAM used by the permanently resident parts of MCS-DOS, which includes program code, but also space reserved for various buffers and caches. |
| `2,048 bytes reserved for C stack` | 2 kilobytes are reserved for the dynamic allocation of variables used by the C program code. |
| `8,192 bytes cartridge ROM window` | 8 kilobytes are reserved for “banking” in and out sections of program code and data stored on the cartridge ROM as needed. |
| `15,361 bytes reserved for system` | This denotes the amount of RAM used by the C64 and not claimed by MCS-DOS. It includes the KERNAL, the zero page, and the I/O stack. |
| `9,968 bytes free` | The amount of unused memory within the RAM areas allocated by MCS-DOS. |

Note that `MEM` attempts no detection of RAM Expansion Units (REU).

## MKDIR

Alias of `MD`.

## MOUNT

Mounts a disk image.

**Syntax:**

```
MOUNT image [drive:]
```

**Description:**

Mounting a disk image attaches it to a drive, letting you use that image as if it were a physical disk. It is only supported with SD2IEC and Ultimate devices, with MCS-DOS using their respective native firmware implementations. You can perform all regular file and disk operations on a mounted disk

image, under the existing constraints for the respective device types. Blank disk images can be created with the `MAKEDISK` command.

With an SD2IEC, mounting a disk image essentially selects it as the current directory on the SD2IEC drive, replacing the current one. The *drive:* parameter should therefore be omitted. However, MCS-DOS will not let you `CD` into a disk image; `MOUNT` is the only canonical way to access disk images under MCS-DOS. Other content on the SD2IEC device is not visible or accessible until the image is unmounted.

When mounting a disk image file from Ultimate storage, it is explicitly linked to one of the emulated disk drives (8 and 9, or A and B). Ensure that the target drive is enabled.

Supported formats are D64, D71 and D81 on both device types; D41 on SD2IEC (treated as D64); G64 and G71 on Ultimate (subject to firmware support); and DNP on SD2IEC. DHD images (CMD hard drive images) are not supported. Disk image files will show up in a `DIR` listing with their image type as file type (instead of `SEQ`).

The *image* parameter can include full drive and path specifiers. On Ultimate, it will also accept native full FAT names, in case a truncated long file name (>16 characters) is ambiguous. Use `DIR /N` to obtain a listing of FAT names on the current mount. The *drive* parameter should not be used with SD2IEC, as the mount always occurs on the same drive as the SD2IEC itself. On Ultimate, you have to specify the emulated drive number or letter, unless you're already on one such drive.

To "unmount" (unassign) a disk image from a drive, use `UNMOUNT`.

**Examples:**

Mount a disk image on a SD2IEC device:

```
MOUNT "LAZY JONES.D64"
```

Mount a disk image on Ultimate storage to emulated drive A (8):

```
MOUNT BARBARIAN.D64 A:
```

Mount a disk image on SoftIEC drive D (11) while currently on an emulated drive (Ultimate):

```
MOUNT D:\GAMES\WIZBALL.D64
```

## MOVE

Copies the source file to the specified destination, then deletes the source.

**Syntax:**

```
MOVE source destination [/P]
```

**Description:**

The main use for `MOVE` is to relocate a file or group of files (as indicated by a wildcard) from one drive to another. It copies the file(s) to the destination, then deletes the original file(s). If the source and destination are on the same drive, the result is effectively the same as using `REN`.

If *source* is a subdirectory or partition, it will rename it. However, `REN` also works for this.

**Examples:**

Move a file from the current drive to drive B (9):

```
MOVE README.TXT B:
```

Move a group of files from drive B (9) to drive A (8):

```
MOVE 9:INVOICE* 8:
```

Rename a file or directory:

```
MOVE INVOICE.TXT INVOICE2.TXT
MOVE PICTURES PHOTOS
```

## PAUSE

Suspends processing of a batch program and waits for a key to resume.

**Syntax:**

```
PAUSE [message]
```

**Description:**

`PAUSE` provides a way to pause execution of a batch file and wait for user interaction. Unless a custom message is specified, it will echo `Press any key to continue . . .`

## PRINT

Sends the contents of a file to a printer.

**Syntax:**

```
PRINT filename [4:|5:|LPT1|LPT2]
```

**Description:**

Default Commodore 64 printer devices are 4 and 5. MCS-DOS also accepts the DOS-style aliases `LPT1` and `LPT2`. If a printer is connected to either one of these ports, the contents of the specified file are sent to it for printing.

`PRINT` also works with the Ultimate 64/C64 Ultimate virtual printing feature, which outputs a PNG image or an ASCII or PETSCII text file. Refer to the Ultimate documentation for setup instructions.

**Examples:**

Send a file to a printer:

```
PRINT CONCEPT.TXT 4:
PRINT NOTICE.DOC LPT2
```

## RD

Removes a directory or partition.

**Syntax:**

```
RD [drive:][path] [/S [/P]]
RMDIR [drive:][path] [/S [/P]]
```

**Description:**

`RD` or `RMDIR` removes a subdirectory or partition on drives that support them. See chapter 5, “Directories and partitions”, for more information.

The `.` or `..` symbols, which respectively denote the current directory or the parent directory, cannot be used. The subdirectory to be removed must also be empty, although you can force the simultaneous deletion of all its contents with the `/S` switch (except on CMD drives).

On CMD drives, deleting partitions (i.e. first-level subdirectories) with `RD` is not supported; you must use HD-TOOLS or FD-TOOLS instead.

The following switches are available:

| | |
|---|---|
| `/S` | Removes all subdirectories and files in the specified directory as well. |
| `/P` | Suppresses the confirmation prompt with `/S`. |

**Examples:**

Remove a subdirectory from the current path, or at another location in the directory tree:

```
RD TEMP
RD /DOCUMENTS/ARCHIVE
```

Remove a subdirectory including all files and subdirectories it contains:

```
RD PROGRAMS /S
```

## REBOOT

Reboots the environment.

**Syntax:**

```
REBOOT
```

**Description:**

`REBOOT` restarts your MCS-DOS session. It skips the bootsplash and the system test, but does execute your `CONFIG.SYS` and `AUTOEXEC.BAT` if present. This can be useful when testing configuration changes.

## REM

Records comments in a batch file.

**Syntax:**

```
REM [comment]
```

**Description:**

`REM` is used to annotate your batch scripts; it doesn't perform any action itself. You can also use REM without comment to add spacing for clarity. Nevertheless, as with all commands, the command is echoed during execution of a batch script, unless `ECHO` is `OFF`.

**Example:**

The following batch program contains `REM` statements to add a functional description:

```
@ECHO OFF
REM SAVESTATS.BAT: Runs a check of the
REM disk in drive A and saves the
```

```
REM report.
REM
CHKDSK A: > A:\DISKSTATS
TYPE DISKSTATS
```

## REN

Changes the name of a file or directory.

**Syntax:**

```
REN [drive:][path][filename1] filename2
RENAME [drive:][path][filename1] filename2
```

**Description:**

The `REN` command changes the name of a file or subdirectory, identified by *filename1*, to *filename2*. Wildcards are not supported. You can rename files on a different drive, although in that case both drive identifiers must match.

**Examples:**

```
REN EDITOR.C EDITOR-OLD.C
REN A:\ADDRESSES A:\ADDRESSES2
```

## RENAME

Alias of `REN`.

## RMDIR

Alias of `RD`.

## RUN

Executes a program.

**Syntax:**

```
[drive:]filename [/A address]
```

**Description:**

`RUN` launches a batch file or a native C64 program from MCS-DOS. However, starting with version 2.1, you may also simply type the program or batch name without the command.

While batch files run fully within the shell and guarantee resumption of control after completion, native programs may not do so. Many programs do not actually implement an exit, and for those that do, it is a requirement that they use the standard BASIC return at addresses `$0302-$0303`, and that they do not overwrite the `$C000–$C1FF` area, which contains the MCS-DOS shell re-entry hook. Otherwise, perform a reset to start a new MCS-DOS session.

When the shell transfers control to an external program, the current shell state (including configuration, history, and internal caches) is saved to the cartridge, and restored if the MCS-DOS return hook is successfully reached. If restoration fails for some reason, default shell settings will be applied.

Batch files are recognized by existence of a `.BAT` extension as part of their filename.

The following switches are available:

| | |
|---|---|
| `/A` | Use for machine code programs without a BASIC stub (`SYS <address>`). The program will be loaded at the specified decimal address, and is jumped to. |

**Examples:**

Run a program:

```
RUN "MONTY ON THE RUN"
RUN 8:CALC
```

Run a program at a specific address:

```
RUN TESTPROG /A 49152
```

## SET

Displays, sets, or removes environment variables.

**Syntax:**

```
SET [variable=[string]]
SET /ENV
```

**Description:**

MCS-DOS provides a 256 byte *environment* to store user-assigned variables in. The environment is normally used to configure shell settings from an `AUTOEXEC.BAT` startup file, which are processed and applied once execution finishes. To this end, MCS-DOS reserves a number of environment variable names – see chapter 7 for more information. Variables with custom names are not rejected, but do not currently have any use.

Variable names have a maximum length of 8 characters, while values can be up to 32 characters long. Settings are case-insensitive, and whitespace is ignored. A variable can be cleared by omitting the value.

Using SET without parameters shows a list of variables currently present in the environment.

The following switches are available:

| | |
|---|---|
| `/ENV` | Displays environment usage statistics. It shows the total environment size, the number of bytes used, and remaining environment space. |

**Examples:**

Configure predefined shell settings (see chapter 7):

```
SET CHARSET=CGA
SET COLOR=3,6,6
SET DIRCMD=/ON
SET DRIVEIDS=DOS
SET PROMPT=$P$G
```

Clear an existing setting:

```
SET DIRCMD=
```

Show environment statistics:

```
SET /ENV
```

Show a list of current variables:

```
SET
```

## SPLASH

Shows the MCS-DOS splash screen.

**Syntax:**

```
SPLASH
```

**Description:**

`SPLASH` reproduces the boot splash screen displayed when MCS-DOS is (re)started. It displays an MCS logo along with the MCS-DOS version and a copyright notice.

## SYSINFO

Provides technical information on your computer.

**Syntax:**

```
SYSINFO
```

**Description:**

`SYSINFO` reproduces the system report that is displayed at MCS-DOS startup, after the bootsplash. It performs various tests to distinguish computer model, display mode, Ultimate board if installed, the KERNAL version, and any connected drives on device numbers 8 to 30. Other properties are inferred from the detected computer model.

The table below lists the properties that are detected by `SYSINFO` and their possible values.

| | |
|---|---|
| **Computer model** | Commodore 64, Commodore SX-64, Commodore 128, Commodore 128DCR, Commodore 65, and MEGA65. C64C and C128D cannot be easily and reliably distinguished from comparable models. |
| **RAM** | Inferred from computer model: 64K or 128K. REU is not detected due to register overlaps between EasyFlash and REU units. No attempt is made to detect “virtual” (Ultimate/emulated) units. |
| **CPU** | Inferred from computer model:<br>MOS 6510/8500 — Commodore 64 (8500 is 64C)<br>MOS 6510 — Commodore SX-64<br>MOS 6510 (FPGA) — Ultimate 64 family (incl. C64 Ultimate)<br>MOS 8502 — Commodore 128 family<br>CSG 4510 — Commodore 65<br>45GS02 — MEGA65 |
| **Board** | Shows Ultimate model if one is detected: Ultimate 64, Ultimate 64 Elite, Ultimate 64 Elite-II, and Commodore 64 Ultimate. The latter may also appear simply as “Ultimate 64”. Note that the Ultimate Command Interface must be enabled for a successful detection. |

| | |
|---|---|
| **KERNAL** | Revisions 1 through 3, or SX-64. Some custom KERNALS are also recognized: JiffyDOS, JaffyDOS, DolphinDOS, SpeedDOS, EXOS. |
| **Video** | VIC-II (C64), VIC-II E (C128), VIC-III (C65), VIC-IV (MEGA65). It is additionally noted whether PAL or NTSC mode is in effect. |
| **Disk drives** | CBM 1541, 1541-II, 1570, 1571 and 1581 models, CMD-FD and HD, SD2IEC and SoftIEC (Ultimate). For CBM drives, physical size (3.5” or 5.25”) along with their storage capacity, density and sidedness are displayed as well, but these are hardcoded per model, not physically detected. |

## TAPECOPY

Loads and transfers one or more files from tape to disk.

**Syntax:**

```
TAPECOPY [filename] drive:
```

**Description:**

`TAPECOPY` implements a tape (Datasette) loader that seeks and loads files, and presents the option to save them to the target drive. Specifying a file name will seek forward until that file is encountered. After agreeing or dismissing to save a file, the loader will continue to seek for new files unless the user deliberately exits, or the end of the tape is reached. `TAPECOPY` does not support custom tape loaders and will terminate if non-standard headers are encountered; however, this is not a guarantee as custom loaders may not set a custom identifier. Any custom loader code is not executed.

To report progress, seek and find messages are displayed as in C64 BASIC, i.e.:
`SEARCHING`
`FOUND <FILE>`

Executable files (PRG) must be fully loaded into memory to allow for saving. The shell therefore suspends just as with entering BASIC or running native programs, with saved state restored afterwards. As parts of MCS-DOS must stay resident during the process, a file size limit of approximately 44 KB applies. Sequential data files (SEQ) are loaded into a buffer allocated in free RAM space, with segments written to the target disk, appending until transfer is complete.

**Examples:**

Seek to the first file on tape and save to drive A (8):

```
TAPECOPY A:
```

Seek to a specific file called “SAVEGAME” and save it to drive A (8):

```
TAPECOPY SAVEGAME A:
```

## TYPE

Displays the contents of a file.

**Syntax:**

```
TYPE filename [/H[:n]| /T[:n]] [/HEX] [/C]
```

### Description:

By default, TYPE displays the specified file as PETSCII text. For binary files, use the /HEX option to enable a hexadecimal viewing mode. TYPE will automatically paginate if the length of the file exceeds the current screen. You can press RUN/STOP to terminate.

The following switches are available:

| | |
|---|---|
| /H[:n] | Shows the first *n* lines of a file (head). |
| /T[:n] | Shows the last *n* lines of a file (tail). |
| /HEX | Displays the raw bytes of a file, rendered as hexadecimal values. |
| /C | Specifies a number of characters or bytes instead of lines when used with /H or /T. |

If *n* is not specified, 10 lines are assumed (10 lines of 8 bytes for hex view).

The hex viewer displays eight bytes per line, displayed as hexadecimal values, and preceded by a six-digit offset. This is followed by a text representation of these bytes with non-displayable characters rendered as dots (.).

### Examples:

Show the contents of a text file called README:

```
TYPE README
```

Show the first 5 lines:

```
TYPE README /H:5
```

Show the last 5 lines:

```
TYPE README /T:5
```

Show the byte contents of a file called PROGRAM:

```
TYPE PROGRAM /HEX
```

Hex view works with head and tail options:

```
TYPE PROGRAM /HEX /H:5
TYPE PROGRAM /HEX /T:5
```

Show the first 40 characters/bytes of a text or binary file:

```
TYPE README /H:40 /C
TYPE PROGRAM /HEX /H:40 /C
```

## UNMOUNT

Unmounts a disk image.

### Syntax:

```
UNMOUNT [drive:]
```

### Description:

UNMOUNT will unlink a mounted disk image from its drive. On an SD2IEC drive, this restores the regular SD2IEC file system. On Ultimate, it “ejects” the image from the emulated floppy drive.

You can omit the *drive* parameter if you’re already on the same drive the disk image is mounted to.

### Examples:

Unmount a disk image from the drive it's mounted to:

```
UNMOUNT 8:
```

Unmount a disk image from the current drive:

```
UNMOUNT
```

## VER

Displays the MCS-DOS version and copyright information.

**Syntax:**

```
VER
VERSION
```

**Description:**

The MCS-DOS 2.1 version string is hardcoded as follows:

```
MCS-DOS Version 2.1
Copyright (C) 2026 MCS Labs
```

## VERSION

Alias of `VER`.

## VOL

Displays the volume label (disk name) and disk ID, if they exist.

**Syntax:**

```
VOL [drive:]
```

**Description:**

If the drive is omitted, the volume information of the current drive is displayed. A volume label can be set with the `LABEL` command, the disk ID with `DISKID`. Both disk properties are also displayed by `DIR` and `CHKDSK`.

# 7. Environment variables

## The MCS-DOS environment

MCS-DOS provides a 512 byte *environment* to store user-assigned variables in. These are formatted as key-value pairs, with a name of up to 8 characters long, and a value with a maximum length of 32 bytes.

The environment is normally used to configure shell settings from an `AUTOEXEC.BAT` startup file, which are processed and applied once execution finishes. This can be done with the `SET` command; see the command reference (chapter 6) for more information. Custom variable names are not rejected, but do not currently serve any purpose. Note that setting or modifying shell configuration variables with `SET` on the command prompt will have no effect.

The following variables can be used to configure the MCS-DOS shell:

## CHARSET

Specifies a custom character set to be used by the shell.

**Syntax:**

```
SET CHARSET=name
```

**Description:**

The value of *name* corresponds with a matching `.CPI` file on the same drive from where the `AUTOEXEC.BAT` is executed, which contains a definition for the character set. This is typically the MCS-DOS cartridge, but alternative startup locations can be set via the `BOOTDRV` directive in `CONFIG.SYS` (see chapter 4). The extension should be omitted.

If `CHARSET` is not set, the shell retains the default C64 ROM font. A standard MCS-DOS distribution ships with `CGA.CPI` and `VGA.CPI`, which reproduce the standard 8x8 IBM CGA and VGA fonts, respectively. Extra character sets can be downloaded from the source code repository: https://github.com/mvanwijk83/mcs-dos/tree/main/extras.

MCS-DOS `.CPI` files are not compatible with MS-DOS files with the same extension (Code Page Information); the CPI extension was merely adopted due to the vaguely similar purpose. The MCS-DOS backronym is “Character Presentation Information”.

Only alphanumeric and punctuation characters are replaced; graphical PETSCII characters remain identical to the C64 ROM set.

**Example:**

Apply the character set defined by `CGA.CPI`.

```
SET CHARSET=CGA
```

# COLOR

Configures the shell colors.

**Syntax:**

```
SET COLOR=c1,c2,c3
```

where:

`c1` = foreground (text) color
`c2` = background color
`c3` = border color

**Description:**

The order foreground, background, border is fixed. All values must be specified, separated by commas. Standard Commodore 64 decimal color numbers must be used:

| | | | |
|---|---|---|---|
| `0` | Black | `8` | Orange |
| `1` | White | `9` | Brown |
| `2` | Red | `10` | Pink |
| `3` | Cyan | `11` | Dark gray |
| `4` | Purple | `12` | Gray |
| `5` | Green | `13` | Light green |
| `6` | Blue | `14` | Light blue |
| `7` | Yellow | `15` | Light gray |

The default setting is `15,0,0`, which is light gray text on a black background with a black border area.

**Example:**

Set a uniform blue background with cyan text color:

```
SET COLOR=3,6,6
```

# DIRCMD

Specifies default parameters to be used by the DIR command.

**Syntax:**

```
SET DIRCMD=[/B] [/L] [/O[sort]] [/Ttype] [/W]
```

**Description:**

You can set the `DIRCMD` environment variable if you prefer `DIR` to use a certain format or sort order by default. For example, you might want to force `DIR` to always sort files alphabetically, or to use lower case, or any combination of options that match your preference. This can be again be overridden on the command line by using the appropriate options in a command.

**Example:**

Always show directory contents with an alphabetical sort order and in lower case:

```
SET DIRCMD=/L /ON
```

## DRIVEIDS

Specify whether to display C64 drive numbers or DOS drive letters.

**Syntax:**

```
SET DRIVEIDS=C64 | DOS
```

**Description:**

Commands that display a drive identifier, such as `DIR`, `CHKDSK`, `LABEL`, or `VOL`, or the prompt if formatted with `$P`, will follow the `DRIVEIDS` setting. The default format, if not specified, is `C64` (IEC device number).

DOS drive letters start with A and map from device number 8 onwards, so that device 8 is A, 9 is B, etc. The user-writeable cartridge storage area is always identified as drive 0 in both modes. `DRIVEIDS` does not force the use of either format on the command prompt; both are always accepted, so e.g. `DIR 8:` and `DIR A:` will always work regardless of the setting.

**Examples:**

Identify drives by their IEC device number:

```
SET DRIVEIDS=C64
```

Identify drives by a DOS-style letter:

```
SET DRIVEIDS=DOS
```

## PROMPT

Sets the format of the MCS-DOS command prompt.

**Syntax:**

```
SET PROMPT=format
```

**Description:**

The format string can contain literal text, as well as the following special codes:

| | | | |
|---|---|---|---|
| `$B` | \| (pipe symbol) | `$P` | Current drive and path |
| `$C` | `:` (colon) | `$Q` | = (equals sign) |
| `$D` | Drive letter | `$R` | Line break |
| `$G` | > (greater-than sign) | `$S` | Space |
| `$H` | \ (backslash) | `$V` | MCS-DOS version number |
| `$I` | Drive identifier according to `DRIVEIDS` | `$$` | Dollar sign |
| `$N` | Drive number | | |

The default setting if PROMPT is not set is `$P$G`, which yields `A:\>` or `8:\>`, depending on `DRIVEIDS` (`0:\>` in all cases for the cartridge storage). The format codes are case-insensitive.

**Examples:**

Mimic a BASIC-style “ready” prompt, with the drive identifier (following the configured format) and a colon on a new line:

```
SET PROMPT=READY.$R$P$C
```

# 8. Output redirection

The screen output of certain commands can be redirected to a file; that is to say, their output is written to a new or existing file instead of displayed on the screen. Supported are `CHKDSK`, `DIR`, `ECHO`, `FIND`, `MEM` and `TYPE`.

To redirect output to a new file, use the `>` operator. If the file already exists, it will be overwritten without asking confirmation. Example:

```
DIR /B > FILES.TXT
```

To append to an existing file, use `>>`. Example:

```
ECHO Call John about the upcoming meeting >> NOTES
```

As with file operations in general, drive prefixes can be added, e.g.:

```
MEM > 9:"MY MEMORY.TXT"
CHKDSK >> A:DISKSTATS.TXT
```

Pagination prompts that would appear when running the command normally from the command prompt do not appear in the redirected text.

Redirection to printers is currently not supported. A workaround is to redirect to a file, and then use the `PRINT` command to send the file's contents to a printer, e.g.:

```
DIR /L /ON > TEMP
PRINT TEMP 4:
DEL TEMP
```

# 9. Batch scripting

Batch files can be useful to automate repetitive tasks that you might find yourself doing in MCS-DOS. Sequences of MCS-DOS commands can be put in a batch file, which can then be run from the command prompt.

## Creating and running batch files

To create a batch file, you can use the built-in editor (launched by the `EDIT` command). Or, if you prefer, or require the ability to create a larger file, use an external text editor. Like native programs, batch files are executed by typing their name on the prompt, or by using the `RUN` command. Batch files must have a .BAT extension as part of their file name to be recognized by the MCS-DOS command interpreter; it must be typed out on the command prompt as well. Since C64 file names have a maximum length of 16 characters – and this includes the extension – this leaves 12 characters to give a proper name to your batch files.

## Batch processing

The commands in a batch file behave the same as if they were being individually ran on the command prompt. For example, changing the active drive in a batch file will persist that state until another drive switch is effected – and remains even after the batch file has finished processing and the prompt is restored.

The default behavior is that all commands in a batch file are displayed on screen while the batch file is running. To suppress it, include an `ECHO OFF` command. To prevent the ECHO OFF command from being echoed itself, prefix it with an `@`, i.e. `@ECHO OFF`.

Any screen output and error messages produced by batch commands are displayed on screen as they also would on the command prompt; the echo setting does not affect this. Press RUN/STOP to terminate if the currently running command supports it.

Some batch file limitations are:

- Batch files cannot call other batch files.
- There are no program-like constructs such as variables, conditionals, labels, or parameters (arguments).

## Example

The following batch script automates a disk duplication procedure. It prompts the user to put a new disk in drive B (9), and asks the user to press a key to continue. The disk is subsequently formatted. The user is then asked to insert a disk in drive A (8). After another key press, the contents of the disk in drive A are copied to the disk in drive B. Finally, a disk status report is saved to the target disk and displayed on screen.

```
@ECHO OFF
REM BACKUP.BAT - formats a new disk in
REM drive B and copies the contents of
REM the disk in drive A.
REM -----------------------------------
ECHO Put the destination disk in drive B
```

```
PAUSE and press a key to continue.
FORMAT B: /Q
BEEP
ECHO.
ECHO Put the source disk in drive A
PAUSE and press a key to continue.
DISKCOPY A: B:
ECHO.
CHKDSK B: > B:DISKSTATUS
TYPE B:DISKSTATUS
BEEP
ECHO Done.
```

## AUTOEXEC.BAT

The main startup configuration file, `AUTOEXEC.BAT`, is a regular batch file. Use it to run any MCS-DOS commands you wish at startup and to configure shell settings. An example `AUTOEXEC.BAT`, named `AUTOEXEC.SAMPLE`, is included on the cartridge.

See “Boot configuration” in chapter 4 for more information, and chapter 7 (Environment variables) for settings you can apply. Execution can be skipped at the bootsplash screen by pressing RESTORE.

# 10. Changelog

## Version 2.1

(dd-Mmm-yyyy)

Expands storage media support, including subdirectories and partitions where applicable. Many refinements to the existing command set and shell behavior.

- Added support for CMD, SD2IEC and Software IEC (Ultimate).
- Added partition and subdirectory support for relevant drive types, including `CD` (`CHDIR`), `MD` (`MKDIR`) and `RD` (`RMDIR`) commands.
- Added disk image support for SD2IEC and Ultimate with `MOUNT`, `UNMOUNT`, and `MAKEDISK` commands.
- Programs can now be run by only specifying their file name, as in classic DOS.
- Added `KEY` command to bind command text to function keys.
- Added new `DIR` switches:
  - `/A`: filter on attributes or directories
  - `/N`: display native FAT names (Ultimate only)
  - `/T`: filter on file type
  - `/S`: recursive listing including all subdirectories
  - The `F` option has been removed from `/O`.
- Added new `FORMAT` switches:
  - `/I`: provide disk ID
  - `/V`: provide volume label (disk name)
  - `/Q`: quick format
  - `/F`: specifies CMD-FD-specific native format types
- Improved head, tail, and hex functionality of `TYPE`:
  - Added `/C` to specify a number of bytes or characters for `/H` and `/T`.
  - `/H` and `/T` now work in combination with `/HEX`.
- `$P` in `SET PROMPT` now denotes the current path; the `DRIVEIDS`-dependent drive identifier has been changed to `$I`.
- `COPY` with no destination will now copy the source file(s) to the current location.
- `MOVE` can be used to rename directories (although `REN` will work as well).
- `PAUSE` can now be made to show a custom message.
- Expanded `SYSINFO` computer model and KERNAL detections, and changed video/display presentation. Also, fixed a bug where Ultimate detection required explicitly re-enabling Command Interface.
- Pressing RUN/STOP on the command prompt will now erase the current input, instead of creating a new prompt.
- Added a safe boot hint on the bootsplash.
- The backslash symbol has been remapped from Shift+SPACE to the pound key (£). The replacement glyph for the standard C64 ROM font, previously identical to PETSCII character $6D/109, has also been modified to be thinner.
- A `VGA.CPI` character set has been added, representing the 8x8 IBM VGA font.
- Optimized RAM usage.
- The documentation `.TXT` files have been replaced with this manual in the ZIP archive.

## Version 2.0

(03-Oct-2026)

Reengineered for EasyFlash cartridge. Improved native program and BASIC support and added new commands.

- Added support for `CONFIG.SYS` to configure boot behaviour.
- Improved native program support: programs with a clean exit to BASIC and that preserve `$C000-$C1FF` will return to the shell.
- The BASIC command replaces `EXIT`. It enters the C64 BASIC environment and implements a wedge (`SHELL`) to return to MCS-DOS.
- Added `DISKINIT` command to refresh drive cache and status, equivalent to the CBM-DOS `INITIALIZE` command.
- Added `TAPECOPY` command to transfer files from tape to disk.
- Added `SYSINFO` command that displays basic system information.
- Added head, tail and hex view options to `TYPE` (`/H`, `/T`, `/HEX`).
- Added `/P` switch to `MOVE` to suppress file overwrite prompt.
- Added `/C` switch to `CHKDSK` to perform a cartridge filesystem compaction on demand. Additionally, the standard `CHKDSK` report now also identifies the drive model and lists the native C64 device number and corresponding DOS letter alias.
- Added `/V`, `/I` and `/Q` switches to `FORMAT`.
- Changed the DOS-style "`R`" file attribute letter for the file lock bit to "`L`", matching actual C64 behavior.
- `DEL`eting files with a wildcard will now prompt for individual confirmation for each matching file, unless `/P` is specified.
- Added file redirection support for `FIND`, and removed it from `HELP`.
- Revised boot sequence with a system information report and a startup message.
- REU test in MEM removed due to register overlap between the cartridge and the REU.
- Revised some system messages.

## Version 1.01

(19-Sep-2026)

Includes several small enhancements.

- Added `/P` switch to `COPY` (overwrite without user confirmation).
- `ECHO` text ending in `;` will omit the line terminator (use `;;` to escape).
- Expanded memory usage report produced by `MEM`.
- Press RESTORE on boot screen for "safe mode" (skips executing `AUTOEXEC.BAT`).

## Version 1.0

(15-Sep-2026)

Definitive public release, considered suitable for general use.

- Fixed a bug where `DEL` did not report failed deletion when a file is locked.

- Removed `/S` switches (omit decimal separators) from `MEM` and `CHKDSK` as they were of limited utility.
- Small memory usage reduction by various code optimizations.
- General code cleanup.
- Finalized inline and offline documentation.
- Finalized release package.
- Source code release.

## Version 1.0 Preview

(13-Sep-2026)

Preliminary limited public release.

# 11. About the author

I grew up with both C64s and old PCs. This project is thus a natural crossover between the computer platforms that were both a big part of my youth. My early tinkering with C64 and PC BASICs was instrumental in eventually becoming a professional software developer.

Having moved on to PCs some time during the mid 90s, I got into C64 emulation during the noughties. During that same decade, I became active in the demoscene (albeit as a PC musician and under a different handle), where Commodore is never far away. I attended several editions of X, among many other events across Europe.

The recent resurrection of Commodore excited me, regardless of arguments of how “real” or legitimate this “new Commodore” is. I bought a C64 Ultimate, which has proven to be far more inspirational than the ability to use emulators on a PC is. Developing MCS-DOS – aided by agentic AI – started as scratching a 30 year old itch, and here we are now. Who knows what else the future may bring.

"MCS" is an alias I used in my early days of hobby programming. It only seemed suitable to bring it back, now that I’m making C64 stuff again. Making it part of the product name makes for a nice pun regarding its relationship to MS-DOS as well.

## Links

**MCS Labs on GitHub**
https://github.com/mvanwijk83

**MCS Labs on itch.io**
https://mcs-labs.itch.io/

# Endnotes

[1] Commodore 65 and MEGA65 in C64 mode may work as well, but this is untested.

[2] FD support, while present, is wholly untested.

[3] Supported on SD2IEC and Ultimate devices. See “Disk images” in chapter 5 for more information.

[4] Although “directory” is not technically a file attribute type, this usage is consistent with `DIR /A` on MS-DOS and Windows CMD.